FUNDAMENTOS DE LA INFORMATICA

Parcial I - 7/10/2014, 12:30 hs.

APELLIDO Y NOMBRE: ULLANOVIC, Diego Martín MATRICULA: 12367

Ejercicio	1	2	3	4	5	Parcial I
Puntaje	2	1	2.5	2	3.5	10
Su evaluación	2	1	2.5	2	3.5	10

1. Dado el siguiente número en base 10 pasarlo a base 2, 8 y 16. Luego reconvertir el número binario encontrado a base 10: $(985)_{10}$

2.- En CODE-2, suponiendo que el contenido del puerto IP4 es H'CBCB ¿cuál es el resultado tras ejecutar las siguientes cuatro instrucciones? Marcar con una cruz su elección en el cuadro.

LLI rD, H'85
LHI rD, H'2B
IN rA, IP4
ST[H'54], rA

a. Almacenar H'2B85 en la posición H'54 de memoria
b. Almacenar H'CBCB en la posición H'54 de memoria
c. Almacenar H'CBCB en la posición H'2BD9 de memoria X
d. Almacenar H'2BD9 en la posición H'CBCB de memoria

3.- Realizar la prueba de escritorio del siguiente código y comentar el significado de cada línea.
a) Qué hace la subrutina? ; b)Si cambio el valor de r7, qué otros datos debo cambiar?; c)Escribir los valores que va tomando la salida OP1

	r0	r1	r2	r3	r4	r5	r7	rD	Comentar
ORG H'0000									Posiciona en H'0000 la carga de instrucciones
LLI r0, H'00	0000								Carga bajo H'00 al registro r0
LLI r1, H'01		0001							Carga bajo H'01 al registro r1
LLI r7, H'06							0006		" " H'06 " " r7
ADDS r2, r0, r0			0000						Hace suma para cargar r2 en 0
start: LLI rD, LO(fin)								0017	Carga baja en rD la dirección de la etiqueta fin
SUBS rA, r7, r2									Resta r7 - r2 para activar el biestable Z
BZ									Salta a fin si valor guardado r2 igual a r7
ADDS rD, r2,r0								0000	Transfiere r2 a rD
LD r3, [H'50]				0003					Carga en r3 el contenido de la dirección H'0050
ADDS r2, r2, r1			0001						Aumenta r2 en 1, a modo de contador
ADDS rD, r2,r0								0001	Suma 0 al r2 para transferir r2 a rD
LD r4, [H'50]					0004				Carga en r3 el contenido de H'50 + rD = H'51
ADDS r2, r2, r1			0002						Aumenta r2 en 1
LLI rD, LO(calc)								0013	Carga en rD la dirección de la etiqueta calc
CALLR									Llamada incondicional a rutina (rD) -> calc
OUT OP1,r5									Muestra por la salida H'01, r5. Promedio de 3
LLI rD, LO(start)								0009	Carga rD con la dirección de la etiqueta start
BR									Salto incondicional a H'0009
fin: HALT									Fin de las instrucciones
calc: ADDS r5,r4,r3						0007			Suma r4 + r3 almacena en r5
SHR r5						0003			Salto a la derecha, activa biestable C
RET									retorno al p.p. dirección contenida en rE.
ORG H'0050									Coloca el puntero en H'0050
DATA H'0003									Guarda en H'0050 el valor H'0003
DATA H'0004									" en H'0051 " H'0004
DATA H'0006									" " H'0052 " H'0006
DATA H'0002									" H'0053 " H'0002
DATA H'0008									" H'0054 " H'0008
DATA H'0008									" H'0055 " H'0008
END									Fin del programa

Solo transcribo las sentencias importantes
lo start siguiente: SUBS r4,r7,r2 = H'0009
BZ = 0000
Carga en r3 y r4 -> H'0003 y H'0004 respect.
los promedia en r5 y los muestra por H'01
luego en el 3er ciclo se promedia H'0006 y H'0008
Al llegar a la 4ta vuelta r7 y r2 valen lo mismo se activa Z y termina el programa

4.- Dado el siguiente código en CODE2, transformarlo para reducir la cantidad de instrucciones y lograr el mismo resultado. Escribirlo en su hoja.

```
ORG H'0100 ; los datos estan en H'0030, H'0031 ==========
        LLI r0,00
        LLI rD,00
        LLI rF,30          X
        LD r1,[rD+30]
        LD r2,[rD+31]
        LLI rF,31          X
        SUBS r3, r1, r2
        LLI rd, lo(uno)
        LHI rd, hi(uno)
        BC
        OUT 01, r2         ; en OP1 (el menor)
        SUBS rA,r1,r2      X
        ST [rD+30],r1      X
        OUT 02,rA          => r3
        HALT
    uno: OUT 01, r1
        SUBS rA,r2,r1
        OUT 02,rA
        HALT
END
```

5.- Programar la serie de Fibonacci. Se debe detener la serie cuando la cantidad de términos llegue a un valor ingresado en una palabra de memoria. P.e.:

En decimal:	0	1	1	2	3	5	8	13	21	34
En hexadecimal:	00	01	01	02	03	05	08	0D	15	22

En este caso se deben generaron 10 términos de la serie y mostrarlos en el puerto OP2.

```
Org  H'0000
     LLI  r0,H'00   LLI r2,H'00
     LLI  r1,H'01
     LLI  rA,H'10
     ADDS r3,r1,r0
ret: ADDS rF,r3,r2
     Out  H'02,rF      2
     ADDS r2,r3,r0,    1 2
     ADDS r3,rF,r0     3
     Subs rA,rA,r1     9
     LLI  rD,lo(fin)
     BZ
     LLI  rD,lo(ret)
     BR
fin  Hall
end
```

r0 -> 0
r1 -> 1
r3 -> 1
r4 -> 2
r3 -> 2
r4 -> 3
r3 -> 3
r4 -> 5

I -> 2
I -> 3 r5

invierto variables

Listado de Instrucciones CODE-2

Codop		Nombre	Nemónico	Parámetros	Formato	Explicación	Ciclos
Bin	Hex						
0000	0	Cargar	LD	rx,[v]	F3	rx ← M(rD+v)	9
0001	1	Almacenar	ST	[v],rx	F3	M(rD+v) ← rx	9
0010	2	Carga inmediata baja	LLI	rx,v	F3	rx(15:8) ← H'00; rx(7:0) ← v	6
0011	3	Carga inmediata alta	LHI	rx,v	F3	rx(15:8) ← v	8
0100	4	Entrada	IN	rx, IPv	F3	rx ← IPv	8
0101	5	Salida	OUT	OPv, rx	F3	OPv ← rx	8
0110	6	Suma	ADDS	rx,rs,ra	F4	rx ← rs + ra	7
0111	7	Resta	SUBS	rx,rs,ra	F4	rx ← rs – ra	7
1000	8	NAND	NAND	rx,rs,ra	F4	rx ← (rs.ra)'	7
1001	9	Desplaza izquierda	SHL	rx	F1	C ← rx(15), rx(i) ← rx(i-1), i=15,...,1; rx(0) ← 0	6
1010	A	Desplaza derecha	SHR	rx	F1	C ← rx(0), rx(i) ← rx(i+1), i=0,...,14; rx(15) ← 0	6
1011	B	Desplaza arit. dcha	SHRA	rx	F1	C ← rx(0), rx(i) ← rx(i+1), i=0,...,14	6
1100	C	Salto	B-	cnd	F2	Si cnd se cumple, PC ← rD	6
1101	D	Subrutina	CALL-	cnd	F2	si cnd se cumple, rE ← rE-1, M(rE) ← PC, PC ← rD	6/10
1110	E	Retorno	RET	-	F0	PC ← M(rE); rE ← rE + 1	8
1111	F	Parar	HALT	-	F0	Parar	6

Binario	Decimal	Octal	Hexadecimal
0000	0	0	0
0001	1	1	1
0010	2	2	2
0011	3	3	3
0100	4	4	4
0101	5	5	5
0110	6	6	6
0111	7	7	7
1000	8	10	8
1001	9	11	9
1010	10	12	A
1011	11	13	B
1100	12	14	C
1101	13	15	D
1110	14	16	E
1111	15	17	F

	4 bits	4 bits	4 bits	4 bits
F0	codop			
F1	codop	rx		
F2	codop	cnd		
F3	codop	rx	v	
F4	codop	rx	rs	ra

base 8:

base 16:

reconvierto al binario a base 10:

② Puerto de entrada 4 => H'CBCB

```
LLI   rD, H'85     > Carga rD con el valor H'2885
LHI   rD, H'28
IN    rA, IP4      -> Carga rA con el valor de la entrada 4 => rA: H'CBCB
ST    [H'54],rA    -> Almacena en [rD + H'54] el valor CBCB
```

rD = H'2885

$\boxed{2BD9}$

③ a) La subrutina realiza el promedio de los valores cargados en los registros r4 y r3. Ese promedio se guarda en r5.

Comentario: Como se realiza en la subrutina el promedio siempre que no sea un resultado entero, se redondea hacia abajo: P.E.: $\frac{3+4}{2} = \frac{7}{2} = 3,5$ en el programa dará 3.

b) Al cambiar el valor de r7 debo tener en cuenta, los posibles valores que tomará r2, ya que si el biestable Z no se activa este programa será un bucle infinito. r7 deberá ser par.
Además también hay que tener en cuenta los valores en la memoria, ya que si r7 aumenta, también debemos agregar valores para obtener resultados deseados. (en cantidad par)

c) La salida tomará 3 valores:
- 0003
- 0004
- 0008

⑤ Programa:

```
        ORG H'0000
        LLI r0, H'00
        LLI r1, H'01
        LLI r2, H'00

        OUT H'02, r0
        OUT H'02, r1
        LLI rA, H'10
        ADDS r3, r1, r0
ret:    ADDS rF, r3, r2
        OUT  H'02, rF
        ADDS r2, r3, r0
        ADDS r3, rF, r0
        SUBS rA, rA, r1
        LLI  rD, LO(Fin)
        BZ
        LLI  rD, LO(ret)
        BR
Fin:    HALT
        END
```

Muestro los 2 primeros términos, no lo supe hacer mejor

En esta linea puedo hacer un LD de una posición de memoria, o requerir un valor por una entrada (IN

Paso a r3 el valor r1

; Muestro el resultado de r3+r2
; Hago una cambio de valores r2 ← r3
; r3 ← rF

Cargo en rA el número H'0010 porque entendi que debo mostrar solo los primeros 10 dígitos. Si quisiera mostrar más puedo requerir por una entrada un valor cualquiera y cargarlo en rA. El programa sería igual. También podría agregar un llamado a subrutina pero me parece más eficiente de esta manera.

(1) Resumiendo el programa modificado

[illegible]

[illegible]

[illegible]

[illegible]

BZ [illegible]

[illegible]

SUBS [illegible]

HALT

END.

<u>Elimine</u>: LLI rF,30 => No utiliza rF para nada.
LLI rF,30

SUBS rA,r1,rZ => Luego de BC, si no hay acarreo procede a hacer nuevamente la resta, repite una instruc.

ST [H'30],r1 => Vuelve a guardar el mismo valor

en OUT 02,rA modifique el registro de salida al no hacer nuevamente la resta.